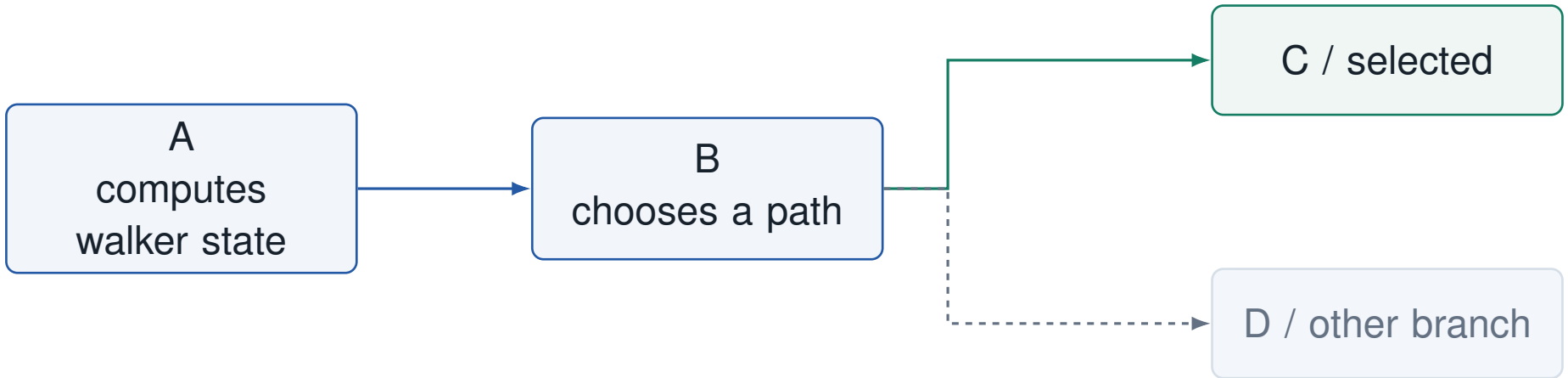
TTG: Dependency-Guided Prefetching in Jac

Use cross-ability dependencies to decide which future graph accesses are ready to prefetch.



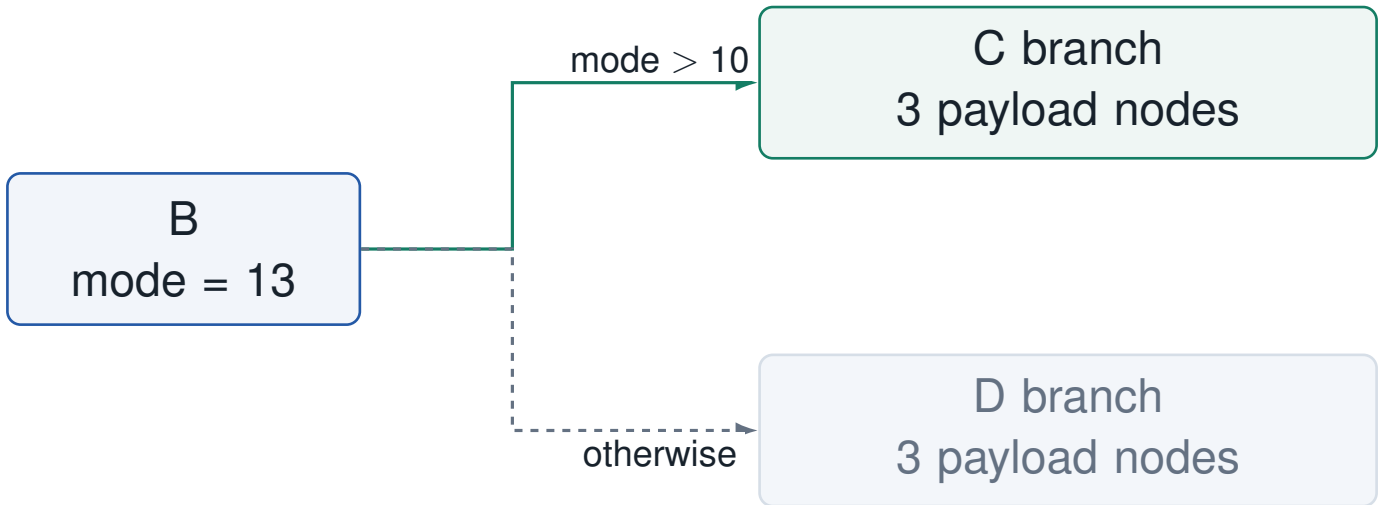
Temporal Trace Graph (TTG)

Static dependency graph + automatic compiler hooks + runtime queue certificates

Selected-visit SQL and real raw-cache consumption are implemented and validated.

A reachable branch can still be unnecessary

A traversal shape can expose both C and D, while a runtime value selects one.

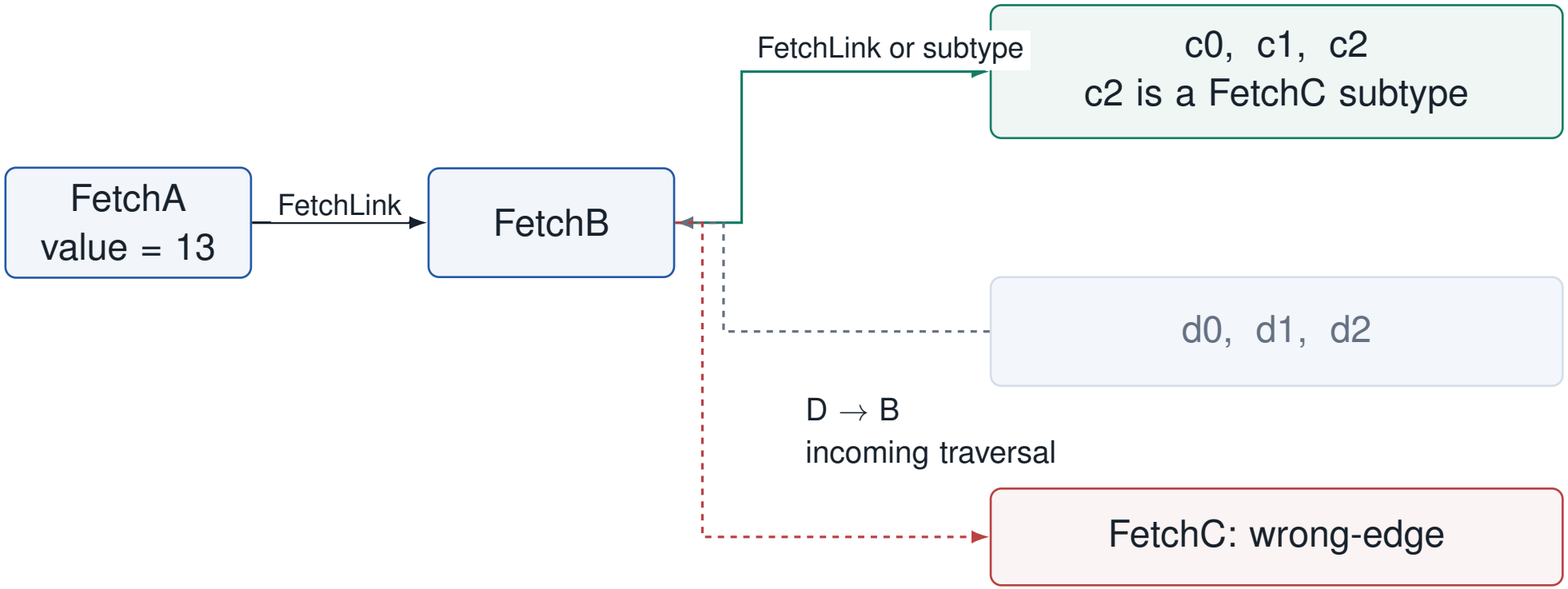


Research question: when can the runtime commit to the selected access path early enough to overlap its data loading with intervening execution?

Predicting a branch requires both a value and a guarantee that intervening work preserves it.

The persisted test graph

Each C/D node stores a label. A different edge type leads to a decoy C node.



Correct prefetch must preserve node type, edge type, subtype matching and direction.

One walker, two entry abilities

A: compute state and enqueue B

```
can start with FetchA entry {
    self.mode = here.value;
    visit [->:FetchLink:->[?:FetchB]];
    self.counter = self.counter + 1;
    if self.counter < 0 {
        visit [-->[?:FetchNoise]];
    }
}
```

B: choose a typed visit

```
can choose with FetchB entry {
    if self.mode > 10 {
        visit [->:FetchLink:->[?:FetchC]];
    } else {
        visit [<-:FetchLink:<-[?:FetchD]];
    }
}
```

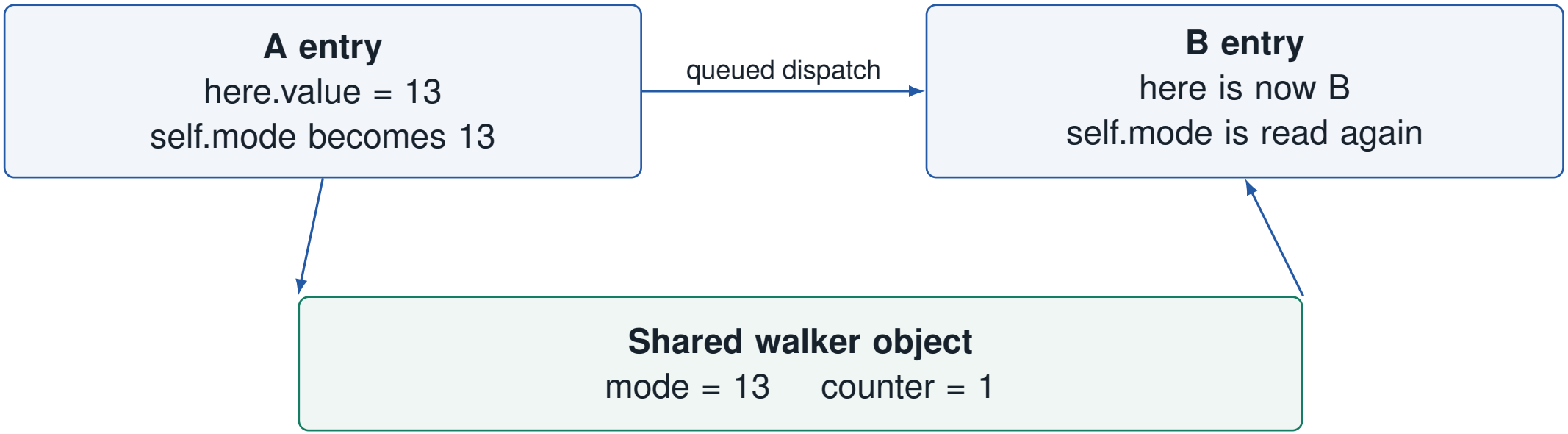
Same FetchBranch instance:
mode, counter, seen

Different current node:
here = A, then here = B

The code shown here is included directly from the PostgreSQL test fixture.

here moves; the walker state travels

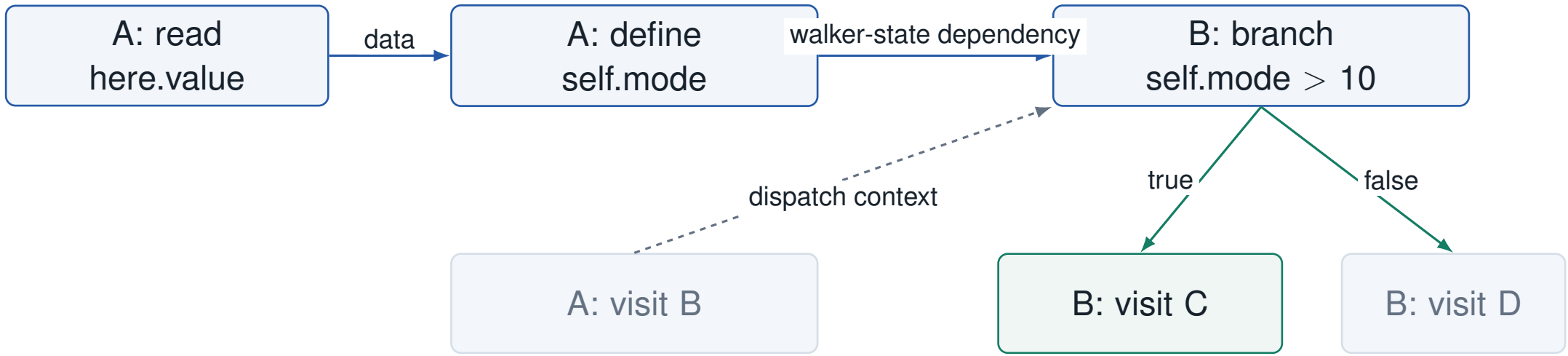
The dependency crosses an ability boundary through the walker field, even though each ability has a different current node.



The compiler connects A’s definition of self.mode to B’s branch input.

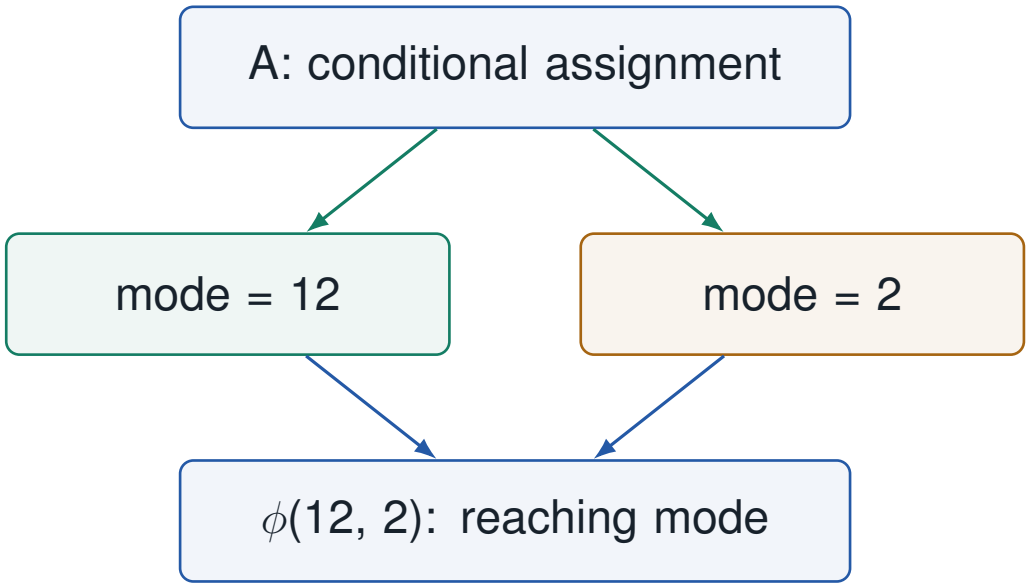
Build a dependency graph over statements

Events represent inputs, assignments, branches, visits and merges. Edges represent data, control, order and cross-ability state transfer.



A state-transfer edge is associated with a specific visit site.

Follow definitions through control flow



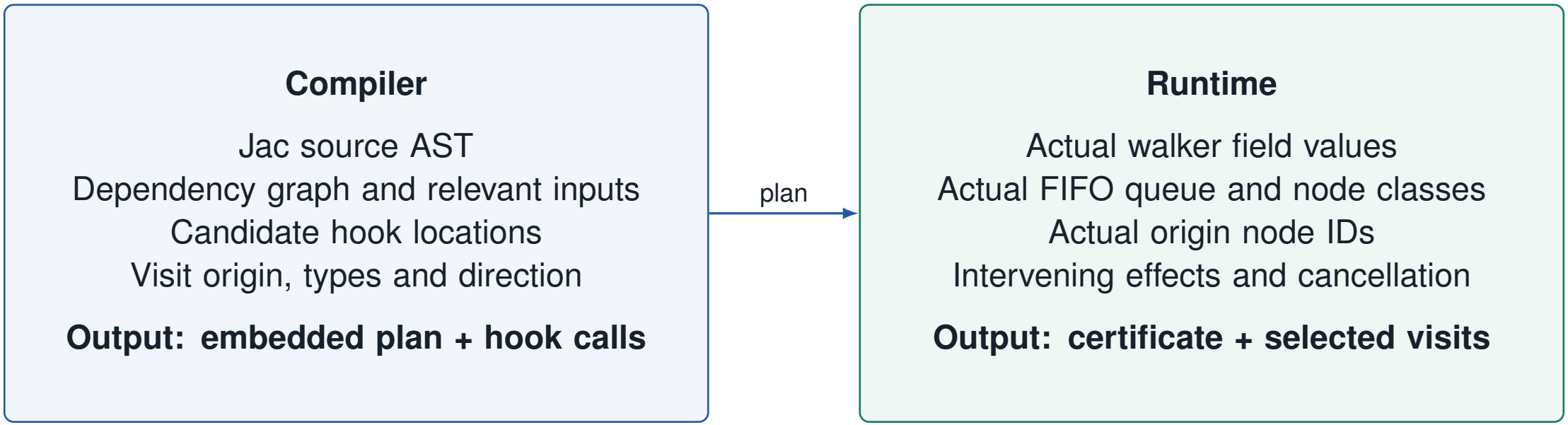
An overwrite kills an old input

B: mode = 0
B: if mode $>$ 0 ...

B's guard then depends on its own definition, rather than A's incoming mode.

Backward slicing tracks definitions, control edges and merges to find the entry inputs a trace needs.

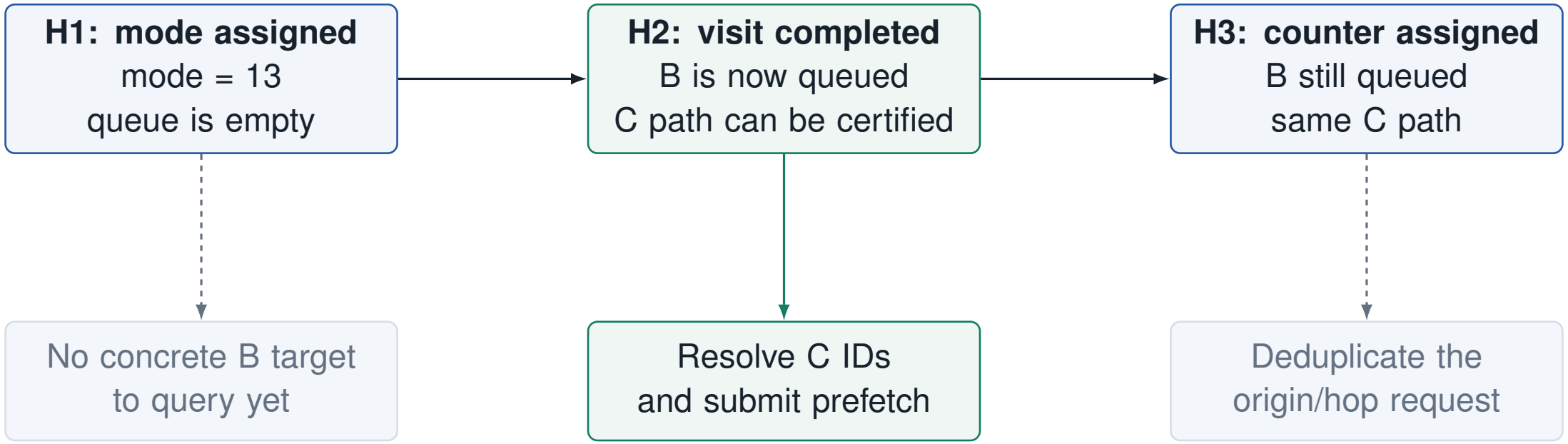
Split the work by available information



The runtime decodes an embedded plan once per capture/class; it uses that plan at each reached hook.

Three candidate points in the A ability

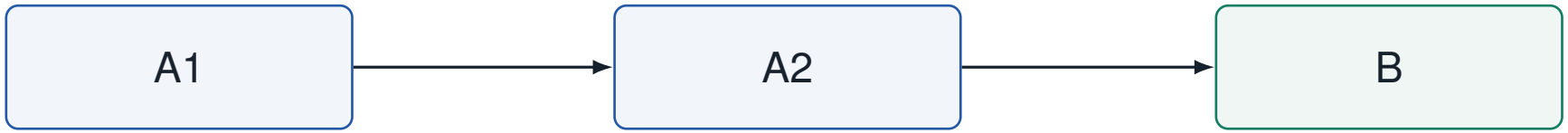
Hooks are inserted after dependency-relevant walker-field assignments and supported typed visits.



counter controls A's other visit, so it gets a hook; it does not change B's mode dependency.

B can be predictable before reaching the head

Start with the actual pending queue:



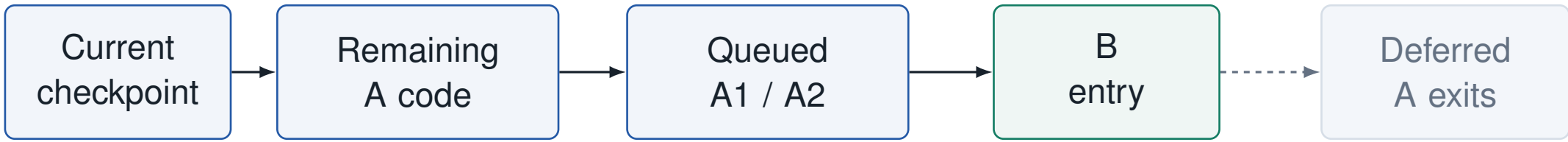
B needs
R = {self.mode}

A1/A2 may write
W = {self.counter}

$R \cap W = \varnothing$: unrelated writes preserve the inputs needed by B's trace.

Analyze the interval before B’s entry

The analyzed interval includes the current continuation, earlier queued entry handlers and handlers preceding the target ability.

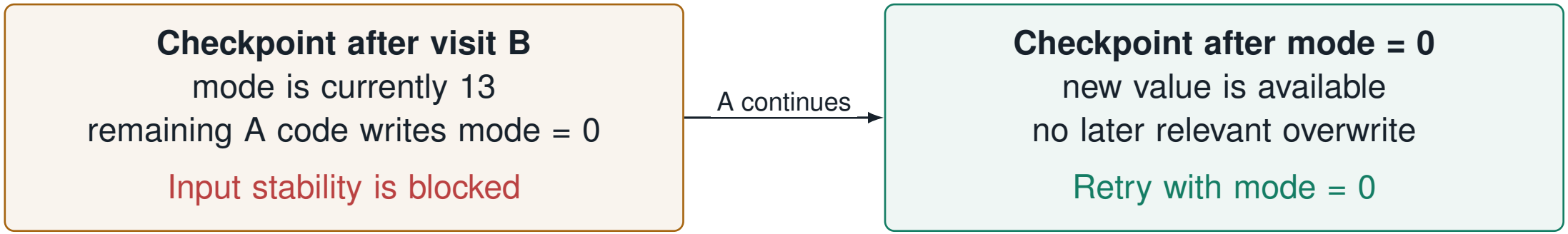


Collect potential writes and unknown effects here.

Exit writes occur after B entry in this FIFO kernel.

Default visits append at the tail: new work from A1/A2 goes after an already-queued B.

A visit schedules B; A's continuation still runs



Validated overwrite fixture:

```
if self.mode > 0 { visit [->[?:DepC]]; }
```

At B entry, mode is 0 and that visit is skipped.

A checkpoint uses the remaining control-flow graph, not just the current value.

Three independent properties

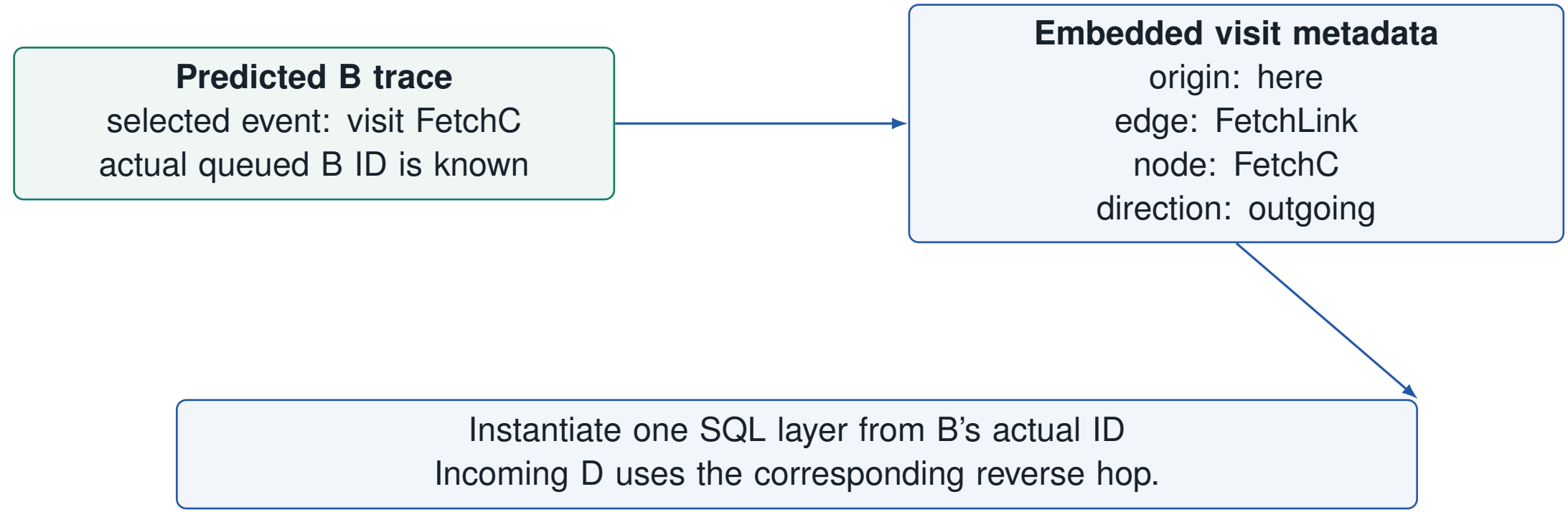
Property	Question	Example
stable	Can intervening code change the required inputs?	counter writes preserve mode
path_determined	Do available values resolve the target ability's trace?	mode = 13 selects C
execution_preserved	Can analyzed preceding work cancel the target?	disengage blocks admission

All three hold → resolve selected visits

Known branch + possible cancellation → skip prefetch

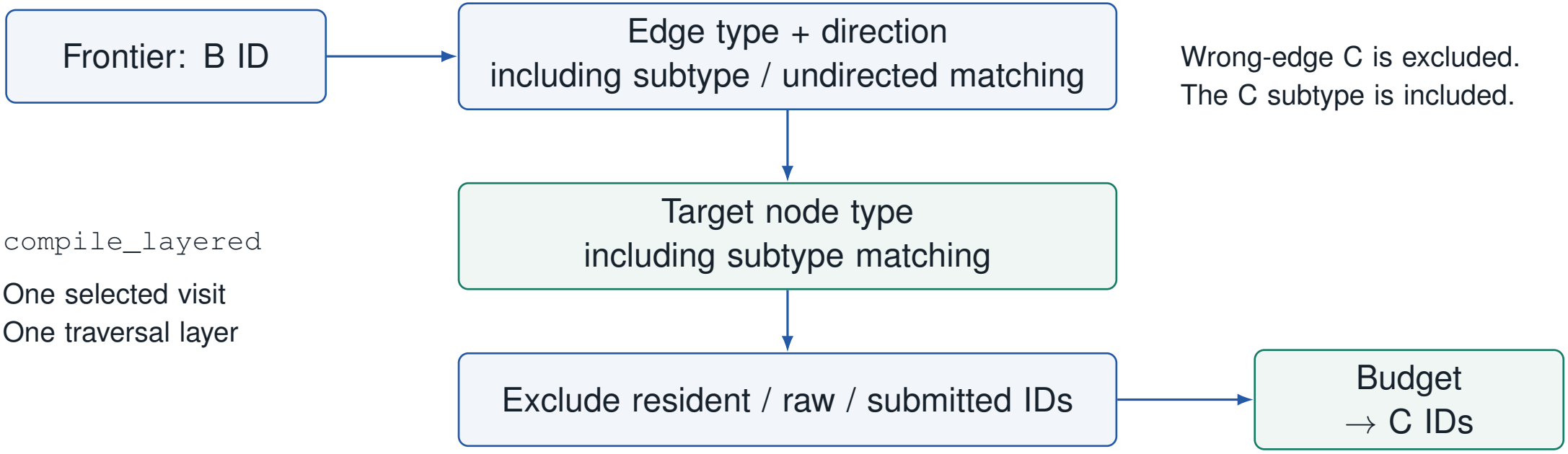
Path knowledge is conditional on execution; cancellation is checked separately.

A selected visit carries its access shape



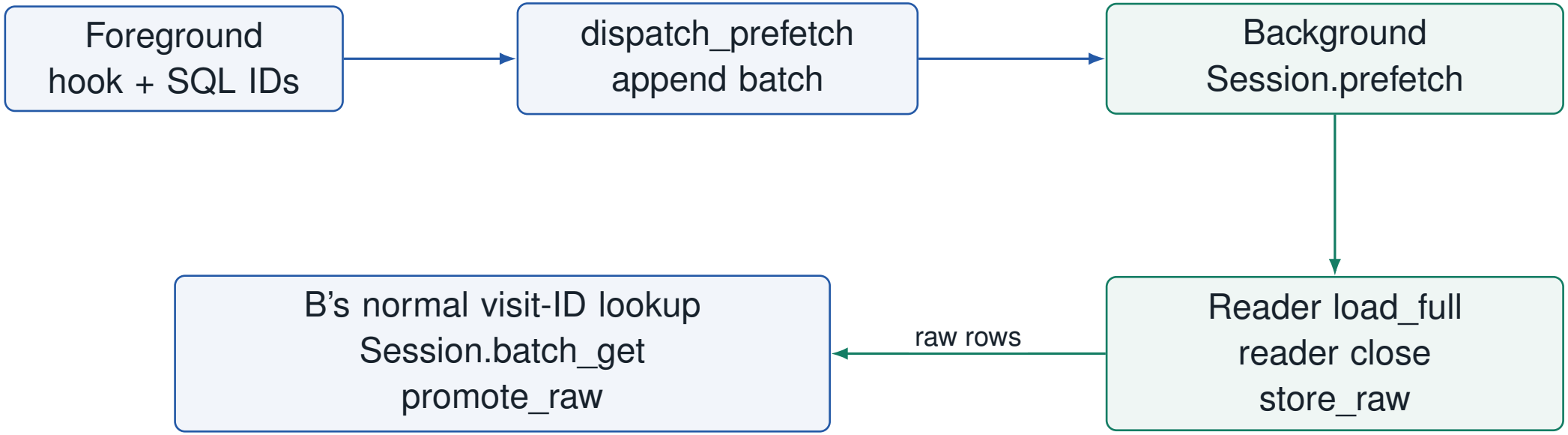
The type-level prediction selects the visit; graph lookup determines the concrete destination IDs.

Reuse the existing layered SQL template



The selected-ID query preserves the visit's access shape and returns the IDs to load.

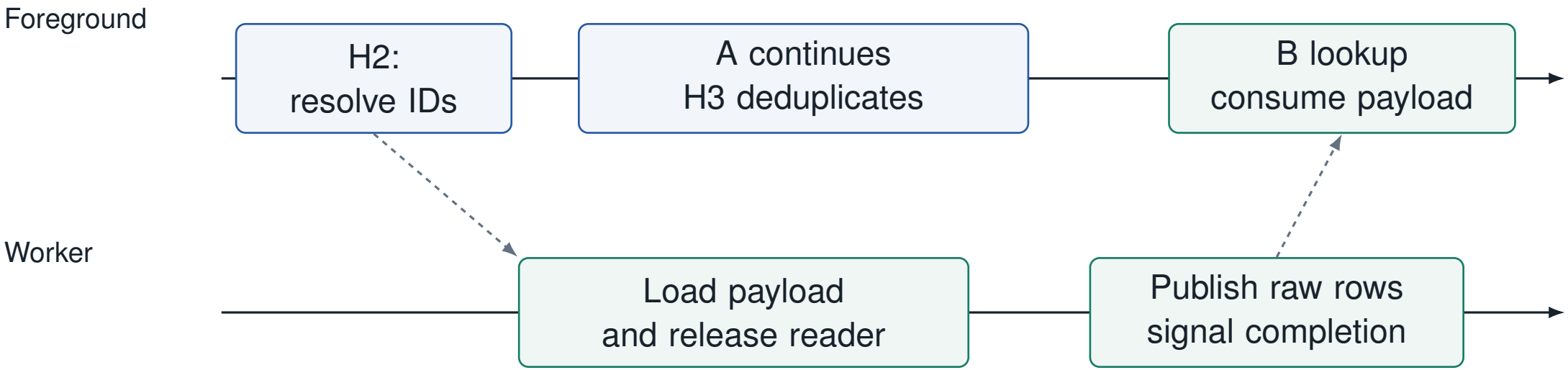
Resolve IDs, load payloads, then consume



Promotion records that the foreground actually consumed a requested row from the raw cache.

The opportunity is an overlap window

Illustrative ordering; horizontal distances do not represent measured durations.



Demand waits within the existing prefetch wait budget, then uses the normal fallback if needed.

Repeated hooks share a bounded plan

Request key
walk scope + origin ID + hop shape

→ H2 submits
H3 finds the same key
one resolution plan

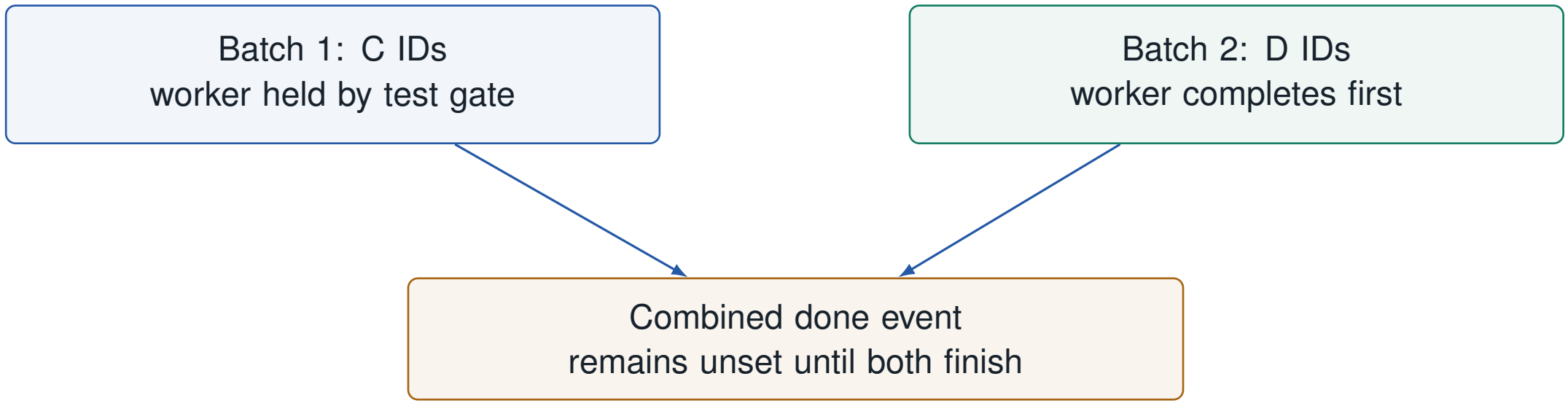
Validated cap = 2
2 IDs submitted
2 raw-cache promotions

Foreground needs 3 results
remaining data uses demand
all 3 labels returned

The cap controls speculative work; ordinary traversal still produces the full result.

Completion covers all outstanding workers

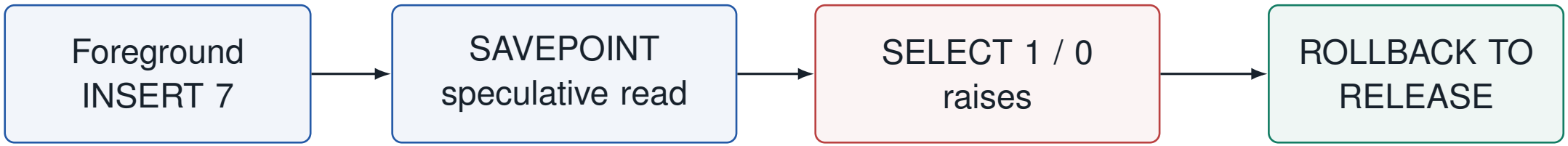
A target ability can select multiple visits. The native two-site fixture requests both C and D.



Thread-order test: both workers are tracked. PostgreSQL test: 6 submitted IDs, 6 real promotions.

A speculative error preserves earlier writes

PgStore.rows_in_savepoint uses the connection under its lock. Session preparation and lookup are serialized by the Session lock.



Validated: the earlier row still contains 7; foreground demand continues.

Count SAVEPOINT / RELEASE and recovery commands when measuring database calls.

Separate submission from actual use

Metric	Meaning
submitted_ids	Unique IDs requested for loading
promoted_ids	Requested rows promoted by the owning foreground thread
unused_cached_ids	Requested rows still in raw cache after workers drain
unavailable_ids	Requested IDs with neither promotion nor a remaining raw row
plans / errors	Selected origin/hops, resolved IDs, status, resolution wall time and foreground errors

Driver traces distinguish worker failure, late fallback and invalidation among unavailable IDs.

Verified PostgreSQL behavior

Case	Submitted	Promoted	Foreground result
Outgoing C branch	3	3	c0, c1, c2
Incoming D branch	3	3	d0, d1, d2
Budget = 2	2	2	all 3 selected labels
Two selected sites	6	6	all 6 C/D labels
Worker failure	3	0	all 3 via demand
Selected-path SQL error	0	0	all 3 via demand

Branch cases submit zero unselected-branch IDs and zero decoy-edge IDs.
Fresh contexts over persisted graphs; assertions inspect real Session promotions.

9 new prefetch cases; 85 targeted runtime tests passed; 11 static and format/lint checks passed.

Historical adapter measurement: 27 October

16 payload nodes per branch, 16 KiB per node. Configured RTT 5 ms; 20 ms independent wait. Medians of 11 trials using an explicit prediction adapter.

Policy	Interval ms	SQL commands	DataRow bytes
Batched demand	39.486	2	271,184
Prefetch both branches	35.544	4	543,120
Dependency-selected	36.834	4	271,936

Selected prefetch removed 16 unused prefetched nodes and approximately halved returned DataRow bytes versus both-branch prefetch. Timing benefits varied across RTT/overlap settings.

This adapter predates automatic hooks and savepoint isolation.
Current-policy timing is a separate measurement stage.

Historical timing depends on RTT and overlap

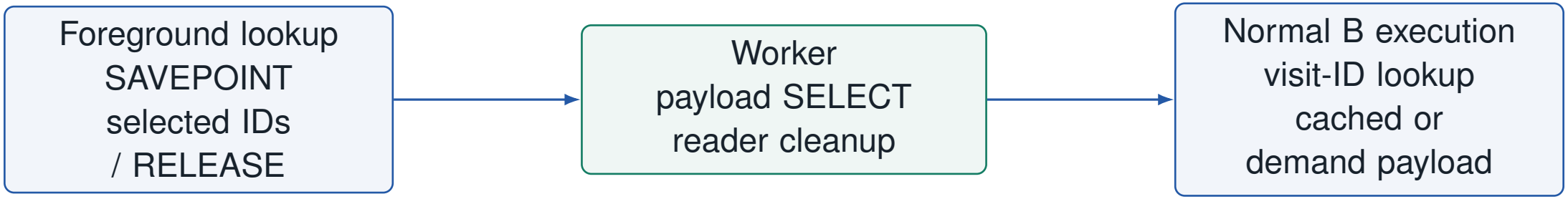
Selected-minus-batched median interval (%). Each cell also shows selected vs. batched milliseconds.
27 October adapter; 11 trials per policy and setting.

	RTT = 0 ms	RTT = 5 ms	RTT = 20 ms
0 ms wait	**+22.3%** 4.854 vs. 3.970 ms	**+34.8%** 21.603 vs. 16.024 ms	**+42.3%** 72.644 vs. 51.054 ms
20 ms wait	**-1.1%** 24.484 vs. 24.768 ms	**-6.7%** 36.834 vs. 39.486 ms	**+4.0%** 72.741 vs. 69.934 ms

Wait models independent I/O. Red: higher selected-prefetch median; green: lower median.
Current-policy evaluation will account for the added hook and savepoint path.

Observed timing varies across settings, motivating measured overlap and cost-based admission.

Account for the complete database path

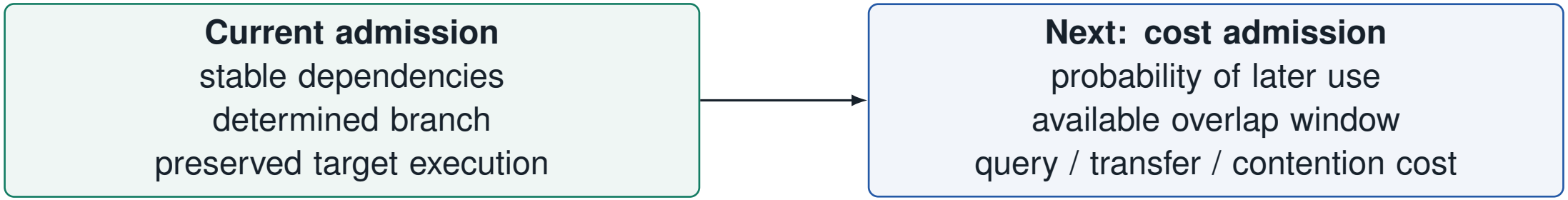


DB-call time
driver entry through response consumption

Foreground wall time
includes computation, waits and overlap

Retain timestamps: concurrent DB-call durations overlap, so their sum differs from critical-path wall time.

Turn a valid prediction into a profitable decision



Measure current-policy DB calls, waiting and unused rows on application workflows.
Use branch frequencies and measured costs to decide when prefetch is worthwhile.

Jac's ability, visit and FIFO semantics supply the context for cross-ability dependency analysis.

Code, configuration and checks

Implementation commit: `9b1ac5ff9`

Branch: `feat/ttg-dependency-graph`

jac.toml / [run]
prefetching = ttg_dependency
async_prefetch = thread
prefetch_limit = 200

Mechanism and API
`docs/design/ttg-dependencies/README.md`

Validation and reproduction commands
`docs/design/ttg-dependencies/validation.md`

Current integration tests
`jac/tests/runtimelib/test_ttg_dependency_prefetch.jac`

Historical adapter results
27 October experiment; preserved medians and driver-boundary method in the deck sources.

GitHub branch: Gorgeous-Patrick/jaseci / feat/ttg-dependency-graph

Source TeX and a reproducible PDF build accompany this presentation.